A SYNTHETIC DATA DETAILS

We generate the synthetic data as follows. We generate a projection matrix $A \in \mathbb{R}^{m \times n}$ with $m < n$, where the elements of A are drawn from a standard Normal distribution, in agreement with the Restricted Isometry Property from compressed sensing. The rows of A are normalized to have unit norm. We generate the latent variables $z \in [0, 1]^n$ with k non-zero components, where the non-zero components are sampled uniformly from $[0, 1]$. The observed variables are then generated as $y = Az$.

When selecting which combinations of latent variables to be active, which k out of n, we consider the particular "out-of-variable" case for OOD generalizations, where some combinations of the variables are not available in the training data. The number of OOD variables is $n/2$. Then, we consider two possibilities:

- **ID data**: Divided into two cases:
  - The first latent variable is active and the other $k - 1$ are drawn between the variables of indices $[2, n/2]$.
  - The first latent variable is not active. The k active indices are drawn from the fool pool of indices $[2, n]$.
- **OOD data**: The first latent variable is active and the other $k - 1$ are drawn between the variables of indices $[n/2 + 1, n]$.

The training set consists of ID data, and the test set consists of OOD data.

B TRAINING DETAILS

We implement the models in PyTorch and train them on a single NVIDIA A100 GPU.

C SPARSE INFERENCE METHODS FOR INTERPRETABILITY

Sparse autoencoders (SAEs) have become the dominant tool for extracting interpretable features from neural network representations (Bricken et al., 2023; Cunningham et al., 2023). An SAE decomposes an activation $\mathbf{y} \in \mathbb{R}^{d_y}$ as $\mathbf{y} \approx \mathbf{D}\mathbf{h}$, where $\mathbf{D} \in \mathbb{R}^{d_h \times d_y}$ is an overcomplete dictionary ($d_h > d_y$) and $\mathbf{h} \in \mathbb{R}^{d_h}$ is a sparse code whose nonzero entries identify the active features. The quality of the interpretation depends entirely on the quality of $\mathbf{h}$: if the codes are wrong, the resulting feature attribution is wrong, regardless of reconstruction fidelity.

Standard SAEs compute codes in a single feedforward pass, $\mathbf{h} = \sigma(\mathbf{W}^\top(\mathbf{y} - b_{\text{pre}}) + b)$, where σ is ReLU or a top-k or JumpReLU activation. This is an *amortised* approximation to the sparse inference problem

$$\mathbf{h}^* = \arg\min_{\mathbf{h}} \ \tfrac{1}{2}\|\mathbf{y} - \mathbf{D}\mathbf{h}\|_2^2 + \lambda\|\mathbf{h}\|_1 \,, \tag{4}$$

which is the Lasso (Tibshirani, 1996), a convex problem with a unique solution (under mild conditions on $\mathbf{D}$). The amortisation gap $\mathbf{h} - \mathbf{h}^*$ is a structured error that is largest precisely when features are correlated or hierarchically organised (Costa et al., 2025; Chanin et al., 2025)—the regimes most relevant to real neural network representations.

Below we compare three inference strategies that move progressively closer to solving Equation (4): FISTA, LISTA, and Matching Pursuit. The comparison focuses on properties that matter for interpretability rather than reconstruction.

C.1 ALGORITHMS

FISTA (Fast Iterative Shrinkage-Thresholding). FISTA (Beck and Teboulle, 2009) solves Equation (4) by alternating a gradient step on the reconstruction loss with the proximal operator for the ℓ_1 penalty (soft-thresholding S_λ), accelerated by Nesterov momentum. Let $\mathbf{h}^{(t)}$ denote the code estimate at iteration t and $\mathbf{q}^{(t)}$ a momentum-extrapolated point:

$$\mathbf{q}^{(t)} = \mathbf{h}^{(t)} + \frac{t_k - 1}{t_{k+1}}\left(\mathbf{h}^{(t)} - \mathbf{h}^{(t-1)}\right), \tag{5}$$

$$\mathbf{h}^{(t+1)} = S_{\eta\lambda}\big(\mathbf{q}^{(t)} - \eta\,\mathbf{D}^\top(\mathbf{D}\,\mathbf{q}^{(t)} - \mathbf{y})\big)\,, \tag{6}$$

where $\eta \leq 1/\|\mathbf{D}^\top\mathbf{D}\|_{\text{op}}$ is the step size. Every iteration updates *all* d_h coefficients simultaneously. The support (which atoms are active) is fluid: a coefficient can be driven to zero by soft-thresholding at step t and revived at step $t' > t$. Convergence to the global optimum is guaranteed at rate $O(1/t^2)$ (Beck and Teboulle, 2009). There are no learned parameters; the algorithm is fully determined by $\mathbf{D}$ and λ.

Practical note. Precomputing $\mathbf{W} = \mathbf{I} - \eta\,\mathbf{D}^\top\mathbf{D}$ and $\mathbf{b} = \eta\,\mathbf{D}^\top\mathbf{y}$ reduces each iteration to $\mathbf{h}^{(t+1)} = S_{\eta\lambda}(\mathbf{W}\,\mathbf{h}^{(t)} + \mathbf{b})$: a single matrix–vector multiply plus elementwise thresholding, both fully batchable on GPU.

LISTA (Learned ISTA). LISTA (Gregor and LeCun, 2010) takes the ISTA update (i.e. Equation (6) without momentum) and untethers its parameters from $\mathbf{D}$. Each layer t computes:

$$\mathbf{h}^{(t+1)} = S_{\theta_t}\big(\mathbf{W}_t\,\mathbf{h}^{(t)} + \mathbf{B}_t\,\mathbf{y}\big)\,, \tag{7}$$

where $\mathbf{W}_t \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{B}_t \in \mathbb{R}^{d_h \times d_y}$, and $\theta_t \in \mathbb{R}^{d_h}$ are *free parameters learned by backpropagation*, independently at each layer. In ISTA, $\mathbf{W}_t = \mathbf{I} - \eta\,\mathbf{D}^\top\mathbf{D}$, $\mathbf{B}_t = \eta\,\mathbf{D}^\top$, and $\theta_t = \eta\lambda\,\mathbf{1}$ for all t; LISTA relaxes these constraints, allowing the network to learn iteration-dependent acceleration. Empirically, LISTA matches FISTA's solution quality in 10–20 layers rather than 100+ iterations (Gregor and LeCun, 2010).

Crucially, LISTA retains the structural properties of ISTA/FISTA: all coefficients are updated jointly at every layer, soft-thresholding provides a continuous sparsity mechanism, and the architecture is fully parallelisable across the batch dimension. The dictionary $\mathbf{D}$ (or its learned analogue in $\mathbf{W}_t, \mathbf{B}_t$) can be trained end-to-end.

MP-SAE (Matching Pursuit SAE). MP-SAE (Costa et al., 2025) unrolls the classical matching pursuit algorithm (Mallat and Zhang, 1993) into a differentiable encoder. Let $\mathbf{d}_j$ denote the j-th column of $\mathbf{D}$. At each step $t = 1, \ldots, T$:

$$j^{(t)} = \underset{j \in \{1,\ldots,d_h\}}{\arg\max}\ \mathbf{d}_j^\top \mathbf{r}^{(t-1)}\,, \tag{8}$$

$$h_{j^{(t)}} = \mathbf{d}_{j^{(t)}}^\top \mathbf{r}^{(t-1)}\,, \tag{9}$$

$$\mathbf{r}^{(t)} = \mathbf{r}^{(t-1)} - h_{j^{(t)}}\,\mathbf{d}_{j^{(t)}}\,, \tag{10}$$

where $\mathbf{r}^{(0)} = \mathbf{y} - b_{\text{pre}}$. One atom is selected per step; its coefficient is computed by projection onto the residual; the residual is updated by subtracting the selected atom's contribution. Previous coefficients are never revised. The dictionary is trained end-to-end via backpropagation through the unrolled steps.

MP-SAE approximately solves a different problem from Equation (4): it targets $\min_{\mathbf{h}} \|\mathbf{y} - \mathbf{D}\,\mathbf{h}\|_2^2$ subject to $\|\mathbf{h}\|_0 \le T$, which is NP-hard; matching pursuit is a greedy approximation with no global optimality guarantee.

C.2 COMPARISON ON INTERPRETABILITY-RELEVANT AXES

Well-posedness of codes. FISTA computes the unique minimiser of the Lasso objective Equation (4). The codes are *defined* by a convex optimisation problem: one can point to the objective and state precisely what the codes mean. LISTA approximates this same solution with learned acceleration. MP-SAE computes the output of a greedy procedure that does not correspond to the global minimum of any fixed objective; the codes depend on the selection order, which is itself a function of the dictionary geometry and the input. For identifiability — where "meaning" is invariance across the equivalence class of valid solutions — the distinction matters: the Lasso solution is unique and characterisable; the MP output is not.

Joint coefficient adjustment. FISTA and LISTA update all d_h coefficients at every iteration. If activating atom i changes the optimal coefficient for atom j (as occurs whenever $\mathbf{d}_i^\top\mathbf{d}_j \neq 0$), subsequent iterations correct for this. MP-SAE sets each coefficient once, at the step the atom is selected, and never revises it. Consider $\mathbf{y} = \alpha_1\mathbf{d}_1 + \alpha_2\mathbf{d}_2$ with $\mathbf{d}_1^\top\mathbf{d}_2 = \rho > 0$. MP selects $\mathbf{d}_1$ first (assuming $\alpha_1 > \alpha_2$) and assigns $h_1 = \mathbf{d}_1^\top\mathbf{y} = \alpha_1 + \alpha_2\rho$, which is inflated by $\mathbf{d}_2$'s contribution leaking through the correlation. The coefficient h_2 computed on the residual is correspondingly deflated. FISTA converges to the correct (α_1, α_2) because it jointly adjusts both coefficients across iterations.

Support dynamics. In FISTA/LISTA, the active set (support of $\mathbf{h}$) is fluid: an atom can be activated, deactivated, and reactivated across iterations as the algorithm converges. This self-correction is critical when the initial support estimate is wrong. In MP-SAE, the support grows monotonically — once an atom is selected, it remains active. There is no mechanism to deselect an incorrectly chosen atom, and the error propagates through all subsequent residuals.

Correlated and hierarchical features. Standard SAEs compute all inner products $\langle \mathbf{d}_j, \mathbf{y}\rangle$ simultaneously and threshold, making all activation decisions in parallel. This implicitly assumes quasi-orthogonality of the dictionary (Costa et al., 2025): if $\mathbf{d}_i$ and $\mathbf{d}_j$ are correlated, activating $\mathbf{d}_i$ should reduce the evidence for $\mathbf{d}_j$, but the one-shot encoder cannot express this.

MP-SAE fixes the conditioning problem via the residual update Equation (10): after selecting $\mathbf{d}_i$, atom $\mathbf{d}_j$ is evaluated against the residual $\mathbf{r}$ rather than the raw input, so correlated atoms no longer double-count shared variance. This is also why MP-SAE naturally recovers hierarchical structure: the first iteration selects the dominant (coarse) feature, and subsequent iterations select progressively finer features on the residual.

FISTA/LISTA handle correlated features correctly *and* with correct magnitudes, because the joint coefficient update avoids the inflation effect described above. However, they do not provide a natural ordering over features — all coefficients converge simultaneously rather than being produced in sequence. When a hierarchy readout is desired, the convergence order or coefficient magnitude in FISTA can serve as a proxy, but the sequential atom selection in MP provides this more directly.

Computational cost. Table 2 summarises the per-step and total cost for a batch of B samples. FISTA and LISTA are fully parallelisable across the batch; MP-SAE's sequential atom selection (the arg max in Equation (8) depends on the previous step's residual) limits GPU utilisation. LISTA compensates for its per-step cost by converging in far fewer steps than FISTA.

Table 2: Computational comparison of sparse inference methods. d_y: input dimension, d_h: dictionary size, T: number of steps/layers, B: batch size.

	FISTA	**LISTA**	**MP-SAE**
Per-step cost	$O(B\,d_h^2)$	$O(B\,d_h^2)$	$O(B\,d_h)$
Typical steps T	100–300	10–20	k (active atoms)
GPU parallelism	Full	Full	Limited
End-to-end trainable	No[4]	Yes	Yes

Trainability. LISTA and MP-SAE are both end-to-end trainable: the dictionary is updated by backpropagation through the unrolled inference steps, using standard deep learning optimisers. FISTA requires alternating optimisation — an outer loop updating $\mathbf{D}$ and an inner loop running FISTA to convergence for each batch — which is slower but provides stronger guarantees on code optimality. A practical middle ground is to train the dictionary using a standard SAE or LISTA, then compute codes at evaluation time using FISTA with the learned dictionary, optionally warm-started from the encoder's output.

D EXPERIMENTAL RESULTS

D.1 METRIC DEFINITIONS AND WHAT EACH DIAGNOSTIC ISOLATES

We use three levels of evaluation metrics, each isolating a different component of the SAE pipeline. Table 3 summarises the distinctions.

Table 3: Summary of evaluation metrics and the questions they answer.

Metric	**Operates on**	**Question it answers**
MCC	Codes vs $\mathbf{z}$ (samples)	Do learned code dimensions track the same variation as the true latents, up to permutation and rescaling?
Accuracy	Probe on codes vs labels	Does a logistic classifier on the codes predict the label OOD? (Same classifier for all methods—fair comparison.)
Per-feature AUC	Codes vs labels (samples)	Does a single code dimension separate the binary label?
Cosine similarity	Decoder $\hat{\mathbf{W}}$ vs $\mathbf{A}$ (weights)	Do the dictionary atoms point in the same directions as the ground-truth columns?
Norm ratio	Decoder $\hat{\mathbf{W}}$ vs $\mathbf{A}$ (weights)	Are the dictionary atoms the correct magnitude?
Support precision	Codes vs $\mathbf{z}$ (per-sample binary)	Of the features the encoder activates, how many are truly active?
Support recall	Codes vs $\mathbf{z}$ (per-sample binary)	Of the truly active features, how many does the encoder find?

Accuracy vs AUC: fairness considerations. The linear-probe baseline uses supervised ridge regression from $\mathbf{y}$ to the ground-truth $\mathbf{z}$, producing d_z-dimensional codes where each dimension is explicitly trained to track one latent. Per-feature AUC directly benefits from this alignment: each output dimension is already optimised to separate its target, giving the linear

[4]FISTA itself is not trained; the dictionary is updated in a separate alternating minimisation step. However, FISTA can be used at evaluation time with a dictionary trained by any method, including an SAE.